Assignment 8 - Shell Variables in Linux

Assignment on the topic of Shell variables in Linux.

You should have the access to Linux box.

Questions for this assignment

1. Explain what shell variables are in the context of Linux. Provide examples of scenarios where shell variables are commonly used.
2. Differentiate between environment variables and local variables. Provide examples of each.
3. Explain how to declare and use shell variables in a script. Provide an example script demonstrating the use of both environment and local variables.
4. Discuss the significance of special variables in shell scripting. Provide examples of commonly used special variables.
5. Explain the role of the **export** command in the context of environment variables. Provide an example illustrating the use of **export**.

Solution by the Instructor:

1. Explain what shell variables are in the context of Linux. Provide examples of scenarios where shell variables are commonly used.
   Shell variables in Linux are placeholders used to store and manipulate data within a shell script or command line. They are symbolic names that represent a value or string, and they play a crucial role in scripting and customization of the shell environment. Shell variables can be categorized into two types: environment variables and local variables.
   Examples of scenarios where shell variables are commonly used:
   1. **Customization:** Users can use variables to customize their shell environment, such as setting the prompt style, colors, or other display configurations.
   2. **Scripting:** In shell scripting, variables are used to store temporary data, making scripts dynamic and adaptable. For instance, a variable may store the name of a file to be processed or the number of iterations in a loop.
   3. **System Configuration:** Variables like PATH and HOME are essential for system configuration. They determine the directories the system searches for executable files and the home directory of a user, respectively.

4. **User Input Handling:** Variables can be employed to store user inputs during script execution, facilitating interactive behavior.

2. Differentiate between environment variables and local variables. Provide examples of each.
   Environment Variables:

- **Definition:** Environment variables are variables that are available to all processes running in the current shell session.
- **Scope:** They have a global scope and can be accessed by child processes.
- **Example:** The PATH variable is an environment variable that contains a colon-separated list of directories where executable files are located.

  Local Variables:

- **Definition:** Local variables are variables with a limited scope, typically confined to a specific shell script or function.
- **Scope:** They are only accessible within the script or function where they are defined.
- **Example:** In a script, a local variable count might be used to keep track of a loop iteration count.

3. Explain how to declare and use shell variables in a script. Provide an example script demonstrating the use of both environment and local variables.
   Declaring and Using Shell Variables:

- To declare a variable, use the syntax variable_name=value.
- To access the value of a variable, prefix the variable name with a dollar sign ($).

  Example Script:

  ```
  #!/bin/bash


  # Environment Variable

  echo "PATH is: $PATH"


  # Local Variable
  ```

```
name="John"

echo "Hello, $name!"


# Modifying a Variable

count=5

echo "Count: $count"

count=$((count + 1))

echo "Updated Count: $count"
```

4. Discuss the significance of special variables in shell scripting. Provide examples of commonly used special variables.
   Significance of Special Variables: Special variables in shell scripting provide information about the script's environment, input parameters, and execution status. They enhance the flexibility and functionality of shell scripts.

   Commonly Used Special Variables:

   1. $0: Represents the name of the script or shell.
   2. $1, $2, ... : Represent command-line arguments passed to the script or function.
   3. $#: Indicates the number of arguments passed to the script.
   4. $@: Represents all the command-line arguments as separate words.
   5. $?: Stores the exit status of the last command.
   6. $$: Contains the process ID (PID) of the current script.

5. Explain the role of the export command in the context of environment variables. Provide an example illustrating the use of export.
   Role of the export Command: The export command is used to mark a variable as an environment variable, making it available to child processes. Environment variables set without using export are only local to the current shell session.

   Example:

```
#!/bin/bash

# Local variable
my_var="Local Variable"

# Exporting as environment variable
export my_var

# Accessing in a child process
./child_script.sh
```

In child_script.sh:

```
#!/bin/bash

# Accessing the exported variable
echo "Variable from parent script: $my_var"
```

In the example, the export command is used to make my_var available to the child process (child_script.sh).